

Instance Segmentation using Mask-RCNN

Team members:

1. Suraj Thapa
2. Abhigya Malla

1. Dataset Description

The dataset used in this project contains three object categories:

- Pen
- Plug
- Shoe

Images were collected manually using a mobile phone camera. Each image contains one or more objects belonging to these categories, captured under varying object orientations to increase diversity. The dataset aims to evaluate object detection and instance segmentation performance using a deep-learning model (Mask R-CNN).

Link to dataset:

https://drive.google.com/drive/folders/1KMMumpIqwH7ESGB86MBXgOCoe4Xpxg1F?usp=drive_link

2. Annotation

Annotations were generated using Roboflow, which provides polygon-based segmentation masks. For each image, the annotation JSON includes:

- Polygon coordinates describing each object instance
- Class label: "pen", "plug", or "shoe"
- Bounding box automatically derived from polygon masks
- Image metadata (height, width, file name)

Structure of Annotation Format

Each annotation follows:

```

{
  "key": "00001.jpg",
  "height": 5712,
  "width": 4284,
  "boxes": [
    {
      "id": "1",
      "type": "polygon",
      "label": "plug",
      "points": [...]
    },
    ...
  ]
}

```

For training Mask R-CNN, segmentation polygons were converted into:

- Binary masks: $N * H * W$
- Bounding boxes (derived from masks)
- Class label indices:
 - 0 = pen
 - 1 = shoe
 - 2 = plug

3. Dataset Partitioning

The dataset was split into 80%:10%:10% for training, validation, and testing purposes, respectively. Splits were generated using shuffled image keys to ensure random distribution of classes.

4. Normalization and Preprocessing

4.1 Image Normalization

Images were transformed into PyTorch tensors and normalized to:

- Convert pixel values to float32
- Scale to [0, 1] range
- Mask R-CNN with FPN handles normalization internally

4.2 Mask Formatting

Segmentation masks were formatted as:

- Boolean tensors: (N, H, W)
- Compatible with TorchVision's Mask R-CNN

4.3 Bounding Boxes

Bounding boxes were stored in XYXY format: [x_min, y_min, x_max, y_max] and wrapped using: BoundingBoxes(..., format='xyxy')

5. Data Augmentation & Transformations

Two sets of transformations were used:

5.1 Training Transformations

To increase dataset variability and model robustness:

Geometric transforms

- Random IoU-based cropping
 - Helps the model learn object localization under partial visibility.
- Resize to training scale
 - Target resolution randomly sampled from [640, 800].
- Square padding
 - Ensure uniform input dimensions for batching.

Photometric transforms

Using TorchVision:

- Random color jitter:
 - Brightness (0.875 – 1.125)
 - Contrast (0.5 – 1.5)
 - Saturation (0.5 – 1.5)
 - Hue (-0.05 – 0.05)
- Random grayscale
- Random equalization
- Random posterization
- Random horizontal flip

Final transforms

- Convert to tensors
- Normalize
- Sanitize bounding boxes (remove invalid or empty boxes)

5.2 Validation Transformations (No Augmentation)

Only:

- Resize
- Pad to square
- Convert to tensor
- Sanitize bounding boxes

Validation uses clean data to measure true model performance.

6. Model and Loss Function

We use Mask R-CNN with ResNet-50 + FPN backbone. Loss Functions Used Internally (Automatically by TorchVision)

Mask R-CNN computes the following:

1. Classification Loss
 - Loss type: Cross Entropy Loss
 - Classifies each proposal as pen, shoe, or plug
2. Bounding Box Regression Loss
 - Loss type: Smooth L1 Loss (Huber)

- Refines localization accuracy
3. Mask Loss
 - Loss type: Binary Cross Entropy Loss
 - Pixel-wise prediction for instance masks
 4. RPN Losses
 - Objectness loss
 - Bounding box regression loss

The total loss is the sum of all components: $\text{Total Loss} = \text{Loss_cls} + \text{Loss_box} + \text{Loss_mask} + \text{RPN_cls} + \text{RPN_box}$

7. Evaluation Metrics

We evaluate:

7.1 Bounding Box Detection Metric: mAP

Mean Average Precision (mAP@0.50)

COCO-style evaluation:

- IoU threshold used: 0.50
- Per-class AP computed for: pen, shoe, plug
- Final mAP = mean of per-class APs

This metric evaluates:

- Localization quality
- Classification correctness
- Confidence calibration

7.2 Segmentation Metric: IoU

Intersection over Union (IoU) is computed between predicted and ground-truth masks: $\text{IoU} = \text{Area}(\text{Intersection}) / \text{Area}(\text{Union})$.

The following were measured:

- Per-class mean IoU
- Mean IoU over true positives

- IoU threshold for TP/FP (0.5)

7.3 Loss-Based Evaluation

Before training, baseline losses were computed:

- Average Training Loss
- Average Validation Loss

These measure the model's starting point and help track progress during training.

8. Baseline Evaluation (Before Training)

Before fine-tuning the heads:

- Backbone = pretrained on
- Classification/mask/box heads = randomly initialized

Observation

- $mAP \approx 0.0$ -> expected for untrained
- Plug sometimes detected with $IoU \approx 0.5$
- Shoe and Pen rarely or never detected
- Baseline loss values provide a reference point for improvements

This confirms that the model must be trained to learn your three custom classes.

9. Conclusion

This project involves training a Mask R-CNN model for segmentation of plug, pen, and shoe objects using polygon-based ground-truth annotations. The dataset was processed with proper normalization, augmentation, and partitioning. Mask R-CNN's loss functions and evaluation metrics such as mAP and IoU were used to quantify model performance.

The baseline results indicate negligible performance before training, which is expected. The next step is to fine-tune the model using the prepared dataset and augmentation pipeline.